

Grounded Control Interfaces for Contact-Rich Robot Agents

Feedback Programs, Position–Force Policies, and Joint Residuals

Research Proposal I · Zimo Wen

September 2026

Abstract

Robot agents can describe a goal and invoke manipulation skills while still failing during contact, obstruction, or precise placement. One potential bottleneck is the action interface: a position target expresses where to move but does not fully express how firmly to make contact, when to yield, or how to compensate for local dynamics. This proposal studies whether task-conditioned control representations improve contact-rich manipulation under matched sensing and interaction budgets. Building on Robot-RSI, it compares three approaches: generating executable feedback programs, predicting task-space position–force and impedance parameters, and learning bounded joint-space residual control. The intended paper will establish when a richer control interface improves both task completion and physical interaction quality, and when simpler feedback is sufficient. Experiments separate the contributions of sensing, action representation, control frequency, and training data, with contact quality and deployment cost evaluated alongside success.

1. Motivation, Research Gap, and Paper Hypothesis

The starting point. Robot-RSI already produces executable skills and numerical motion commands. Its LIBERO end-effector increment skill uses servo control and motion solving to produce joint increments and gripper commands. The research question is therefore which additional control variables a model should choose, from which observations, and at what update rate. It is not the introduction of numerical actions into an otherwise purely semantic agent.

The inspected Flexiv adapter exposes joint state, joint torque, external wrench, and gripper-force parameters. A contact-descent handler stops when the measured force change crosses a threshold. These capabilities provide an entry point, but sensing contact and maintaining a desired contact force are different behaviors. The inspected Cartesian-motion wrapper mainly passes pose and velocity and retains hardware-interface checks. Gripper closure force, contact detection, and a six-dimensional end-effector wrench must remain distinct experimental variables.

Motivating tasks. Placing an object on a surface of uncertain height requires a controlled transition from free motion to contact. Pushing along a surface requires maintaining normal contact while progressing tangentially. Pulling a resistant drawer requires distinguishing friction, approach misalignment, and grasp slip. A feedback program, task-space impedance, or joint residual might solve these cases; lower-level output is not assumed to be universally better.

Central hypothesis. A task-conditioned interface that expresses contact goals and feedback can improve manipulation more efficiently than position commands alone, while joint-level learning is useful only when task-space feedback cannot compensate for the relevant error. This is a self-contained paper about **the appropriate action abstraction for robot agents**. The three approaches are candidate methods and controlled comparisons within that question, rather than unrelated applications.

Prior work and intended novelty. Code as Policies already includes feedback programs and impedance-control examples. Diffusion Policy and π_0 provide continuous action-generation foundations. The proposed increment is to ground control structure or parameters in task intent and actual contact evidence, then identify their generalization boundaries. Adding a force field to an API or using a larger action network alone is insufficient. The main claim must be supported by unseen objects and dynamics, matched observations, and measurable physical improvements.

2. Approach A: Executable Local Feedback Programs

Representation and execution. A model generates a program with branches, loops, and termination conditions that reads visual error, proprioception, grasp state, and available force measurements. A program might approach the visible target, reduce speed after contact, and change a local search direction when progress stalls while lateral force rises. The language model supplies inspectable logic; a local runtime executes feedback at a declared rate without requiring a remote language-model call for every small correction.

Generation contract. Expose typed primitives for reading errors, setting short-horizon pose references, observing contact, changing modes, and terminating motion. Each program declares its coordinate frame, action bounds, maximum duration, update period, and exit conditions. Static validation checks types, unbounded loops, and interface consistency. A single control owner applies commands. These checks establish executability, not physical correctness; candidate programs must still be evaluated inside a simulator feedback loop.

Learning and adaptation. Development traces connect scene conditions, candidate programs, and observed responses. The agent generates and revises programs using development feedback and independently verified outcomes. A key comparison asks whether a fixed feedback structure transfers by updating perception-derived parameters, or whether regenerating the entire program is necessary. Test-time models, prompts, and libraries are frozen. Local program generation may occur within an episode, but test outcomes do not become persistent capabilities for later evaluation tasks.

Mechanism test. Compare fixed skills, generated open-loop programs, handcrafted feedback programs, and generated feedback programs. Open- and closed-loop variants share approach trajectories and primitives as far as possible. The difference should concern observation-conditioned corrections rather than access to stronger tools. If the agent only selects a known template, the contribution is task-conditioned selection and transfer, not discovery of a new control law. Measure recovery from local errors, success, and model-call reduction.

3. Approach B: Position–Force and Impedance Policies

Action representation. Predict $a_t = (\Delta x, \Delta R, f^*, K, D, m)$: pose increments, desired contact force, stiffness, damping, and contact mode. A contact frame specifies normal and tangential directions. Selection matrices assign position control and force control to compatible axes, avoiding contradictory constraints on the same degree of freedom. Units, frames, timestamps, and update intervals are explicit in every action packet.

Local realization. A controller uses pose, velocity, and force error to track these references. For example, a normal-direction admittance loop can adjust displacement from $f^* - f_t$ while tangential motion follows a path. The policy updates references at a lower rate than the device-supported feedback loop. Parameter amplitude and slew-rate limits constrain changes. Contact establishment, maintenance, and separation have distinct transitions. Positive stiffness is an engineering constraint, not a proof of overall closed-loop stability.

Training. Collect successful and unsuccessful development interactions from verified control templates and learn the mapping from instruction and observation to control parameters. Compare template-conditioned prediction with a continuous action-generation head. Training can supervise parameter trajectories and execution outcomes, but model selection depends on physical performance on validation tasks. Sensor bias, frame conversion, and filter delay are recorded so that gravity or inertial changes are not automatically mistaken for contact events.

Expected advantage and limitations. This representation uses low-dimensional physical structure and may need less data than learning all joint actions. Its success can nevertheless depend on contact-direction estimates and template coverage. Compare fixed parameters, per-task manual tuning, and observation-conditioned parameters to isolate online adaptation. A model that outputs one constant parameter set has learned parameter selection, not necessarily state-dependent contact control.

4. Approach C: Bounded Joint-Space Residual Control

Formulation. Retain a validated baseline controller and use a small local action network to predict a corrective joint torque from proprioception, force sensing, and task context:

$$\tau_t = \tau_{\text{base}}(s_t) + \text{clip}(\delta\tau_\theta(s_t, g, h_t), -b, b).$$

Residual amplitude, slew rate, and contact-power limits are fixed after validation. The local network executes in the control path; language reasoning provides slower task context. Token-by-token generation of high-frequency torque by a general language model is not required.

Learning and interfaces. In a simulator that actually supports torque commands, establish baseline behavior before learning residual corrections through demonstrations or reinforcement learning. Randomize load, friction, delay, and sensor bias to discourage compensation for a single simulator defect. The actor only receives deployable observations. Hidden dynamics may define evaluation strata or offline analyses but never become actor inputs. Hardware transfer requires checking control frequency, communication delay, and baseline-controller equivalence.

Action Interfaces for Robot Agents

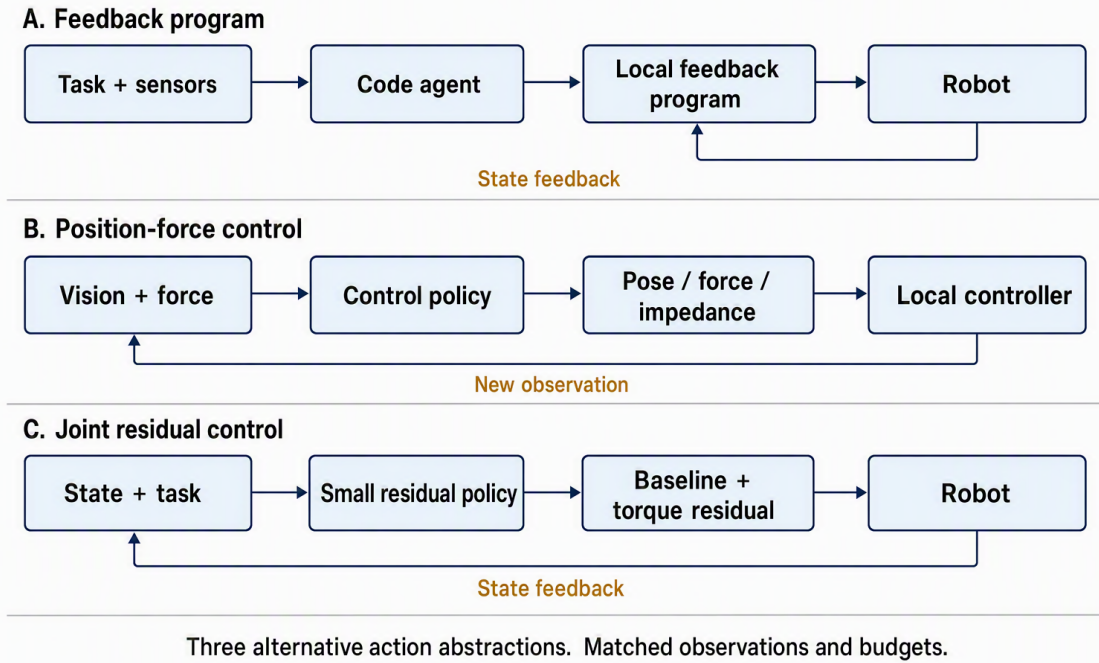


Figure 1: Candidate control interfaces. The three rows produce feedback programs, task-space references, or joint residuals. In row B, robot execution is included in the controller path and observations return from robot sensors. GPT-image-2 conceptual diagram; no measured result is depicted.

Discriminating comparison. Compare no residual, end-effector residuals, impedance adjustment, and joint torque residuals with matched sensing, training data, and approximately matched capacity. If lower-level output helps only for specific disturbances, report that boundary. Retain unstable and saturated trajectories. Clipping a residual constrains execution but does not establish reliability or prove that the learned component caused a successful task.

5. Experimental Design and Evaluation Panels

Standard versus contact evaluation. The standard 120-task LIBERO short panel tests whether the new execution component preserves basic capability. It reports frozen task-level pass@10 by spatial/object/goal subset and task/object/swap/lan suite. A separate contact-control extension evaluates force and torque interfaces, documenting changes to controllers, sensors, and action spaces. Improvements in this extension do not count as standard-benchmark gains.

Main contact panel. Use three task families: contact placement, surface pushing, and drawer interaction. Four object or geometry configurations per family give 12 frozen task instances. Development and validation use different objects, layouts, and physical conditions. Three predetermined disturbance levels and five initial seeds per task yield 180 episodes per method. Each task varies quantities with a clear physical interpretation, such as friction, resistance, or surface height.

First vary friction, load, surface location, and action delay separately; then evaluate held-out combinations, such as surface-height error combined with changed friction. Development verifies that the perturbation range remains solvable using a known effective controller. Greater resistance must not be confused with physical impossibility. Slip and contact-loss criteria use task-appropriate windows and report detection errors.

Comparison	Action and feedback	Matched conditions
Position control	Pose or joint increments	Vision, proprioception, and available force
Contact-stop baseline	Position motion with force threshold	Calibration and execution limits
Handcrafted feedback	Fixed program and tuned controller	Primitives and observation delay
Generated feedback	Conditions and recovery branches	Primitives and model budget
Learned contact policy	Pose, force, and impedance	Training data and contact limits
Joint residual	Baseline plus bounded torque	Observations and approximate capacity

Separating representation from sensing. The main comparison supplies the same force observations to all methods, followed by a no-force ablation. This separates access to better information from the ability to express different control. Record control-update rate, action horizon, and low-level limits. Where architectures cannot be identical, report parameter count, training data, training compute, and deployment latency, with capacity- or latency-matched auxiliary comparisons.

Physical-robot validation. If the Flexiv continuous-force interface and logging chain are verified, use the three matched task families with three physical configurations and ten randomized trials each: an initial 90 trials per method. Calibration and model selection occur before formal trials; training and test objects remain separate. Include joint residuals only if the actual torque interface and controller have been validated. Otherwise, retain torque conclusions as simulation-only and avoid presenting them as successful sim-to-real transfer.

Prespecified endpoints. Final task success is the main behavioral endpoint. Select one task-appropriate contact-quality endpoint before testing, such as peak placement force or force-tracking error during sustained pushing. The panel sizes above are proposed starting points; validation variance determines whether additional task diversity is necessary to support the intended effect-size claim.

6. Planned Experiments and Expected Interpretations

Experiment A: action abstraction. Compare position interfaces, generated feedback, contact parameters, and joint residuals under identical visible observations. Evaluate task success together with interaction quality so that faster completion with a larger impact is not summarized as an unqualified improvement. Retain force and action traces for failed episodes as well as successful ones.

Physical metrics. For contact establishment, report peak force, contact impulse, overshoot, and settling time. During contact maintenance, report force-tracking RMSE, contact-loss fraction, and slip. During motion, report position error, completion time, and abnormal oscillation. Force error is computed only during intervals with a defined target force; including free-space intervals would artificially reduce the average. Report physical units by task before applying any preregistered cross-task aggregation.

Experiment B: contact modes and compliance. Hold the policy structure fixed and remove adaptive stiffness, adaptive damping, mode transitions, or short-term feedback in turn. Compare force-only, compliance-only, and joint prediction. Align parameter traces with contact phases to identify redundant outputs or constant parameter selection. Evaluate whether parameters change appropriately when resistance increases, rather than merely whether the final task succeeds.

Experiment C: whether generated feedback is used. Keep program structure and total action budget fixed while delaying or freezing one feedback channel, or removing the recovery branch relevant to a validated local disturbance. Test whether the program changes behavior after a contact anomaly. Bound full-program regeneration so that results do not rely on unlimited replanning. Attribute failures to perception, conditions, primitives, or interfaces before labeling them language-reasoning failures.

Experiment D: what residuals compensate. Change loads and friction and compare learned residuals with equal-magnitude random residuals, end-effector residuals, and a better-tuned baseline. If the learned residual mainly cancels a constant gravity-compensation error, calibration may be a simpler explanation. Introduce delay and sensor bias to test amplification of high-frequency noise. Report all protective interventions and unstable traces.

Possible result	Defensible interpretation
Feedback improves recovery; contact references reduce force peaks	The interfaces address different errors; explain behavioral and physical gains separately.
Impedance matches torque residuals with less data	Low-dimensional structure suffices; joint-level output is not generally necessary.
Residuals help only under unseen loads	Supports a specific compensation role; examine regression under ordinary conditions.
Success rises alongside oscillation or protection triggers	Overall control quality has not necessarily improved.
Force sensing helps, but action interfaces perform similarly	Sensing may dominate; narrow the representation claim.

Statistics and cost. Use matched seeds and bootstrap intervals clustered by task instance. Randomize hardware method order within trial blocks to reduce thermal and equipment-state effects. Record model calls, tokens, perception/action/recovery time, training volume, and protection-trigger rate. Label secondary analyses explicitly to avoid selecting favorable curves after testing.

7. Feasibility, Risks, and Expected Paper Contribution

An actual interface is a prerequisite. The inspected LIBERO path uses joint increments. Force and torque experiments require implemented, functionally validated controllers. Minimal checks cover free-space tracking, contact with a known surface, coordinate transforms, sensor offsets, mode changes, and command latency. Hardware checks combine SDK semantics with functional trials. These establish that the research variable exists; a working controller is not itself evidence that learning improved behavior.

Stability boundaries. Generated programs, positive impedance parameters, and bounded residuals do not guarantee stability under arbitrary contact. Analyze appropriate local models or passivity assumptions where possible and measure behavior outside those assumptions. Report saturation frequency and the actions actually executed whenever predictions frequently hit limits, allowing readers to distinguish model behavior from the baseline controller’s protective behavior.

Simulation-to-real uncertainty. Friction, contact stiffness, filtering, and tool parameters can differ between simulation and hardware. Document the training randomization range, validation choices, and real test conditions separately. Success with one object does not establish contact generalization. Matched and unseen conditions both require evaluation, and non-transferring failure modes remain part of the result.

Outcome accounting. Only final simulator or independent-harness verdicts count as task success. Infrastructure interruptions, implementation errors, and protection events are reported separately. A method that often fails to execute cannot claim superior reliability from the success rate of its remaining valid episodes. Main tables include scheduled instances, valid verdicts, success/failure counts, infrastructure records, and implementation-error coverage. Preserve videos, force curves, trajectories, diagnostic runs, and exact manifests.

Expected contribution. The paper should establish which errors are addressed by feedback programs, task-space contact references, and joint residuals; introduce a grounded mapping from task intent and contact evidence to executable control; and evaluate that mapping on unseen objects and dynamics. Contact quality, inference delay, and data cost are release criteria alongside success. The contribution is a supported account of when a richer interface is useful, not the fact that an agent can output lower-level numbers.

Independence from the other papers. This study uses a fixed skill library and declared control models. It does not require a new responsiveness algorithm or GPT-guided DAgger training. The candidate approaches share a task and logging protocol but receive separate mechanism analyses. The proposed sample counts and anticipated outcomes are experimental designs, not completed results. Training loss remains a diagnostic; trained policies must be tested in simulation on matched and held-out tasks.

References

- [1] Code as Policies: Language Model Programs for Embodied Control. 2022. <https://arxiv.org/abs/2209.07753>
- [2] Diffusion Policy: Visuomotor Policy Learning via Action Diffusion. 2023. <https://arxiv.org/abs/2303.04137>
- [3] π_0 : A Vision-Language-Action Flow Model for General Robot Control. 2024. <https://arxiv.org/abs/2410.24164>
- [4] Octo: An Open-Source Generalist Robot Policy. 2024. <https://arxiv.org/abs/2405.12213>
- [5] Noematrix Team. RoboRSI research preview. 2026. <https://lab.noematrix.ai/blog/2-roborsi-research-preview/>

Responsive Robot Agents

Event-Triggered Execution, Asynchronous Actions, and Predictive Control

Research Proposal II · Zimo Wen

September 2026

Abstract

A robot agent reasons from observations that are already in the past, while previously issued actions may continue during inference and communication. Target movement, grasp slip, obstruction, or an instruction change can therefore invalidate an otherwise reasonable plan before it finishes executing. This proposal studies responsiveness as the time required for an environmental event to produce an effective change in physical action. It develops and compares three approaches: event-triggered preemptible skills, delay-aware asynchronous action chunking, and predictive control with uncertainty. Robot-RSI provides the initial execution infrastructure. Controlled dynamic perturbations will separate sensing, detection, reasoning, scheduling, and actuation delays, while jointly evaluating recovery, final success, and computational cost. The intended paper asks when execution-time adaptation yields benefits beyond faster inference or more frequent observation.

1. Research Problem and Standalone Paper Claim

A concrete failure. A robot reaches for a cup, but the cup moves during the reach. Producing the next instruction faster does not ensure that the old motion has stopped. A new instruction based on the earlier image may simply execute an outdated action sooner. Grasp slip requires a different response, and a revised user instruction may require a new goal. These events differ in urgency and in what constitutes successful correction.

Starting from Robot-RSI. Some existing motion skills already contain local feedback, and the Flexiv contact-descent routine can detect force internally and stop. These capabilities must be distinguished from task-level adaptation during execution. In the inspected sidecar, motion, state reads, and stop requests share a request lock, so an external observation or stop can wait for the current handler. This finding concerns the software request path, not independent hardware emergency-stop behavior. The research target is timely incorporation of new evidence into execution.

Definition. Let t_e denote the physical event time and t_a the time when a corrective action begins to take effect. End-to-end response latency is $L = t_a - t_e$. The evaluator records t_e without announcing it to the policy. Recovery time and final task success are separate endpoints: immediate stopping may shorten a response measure without completing the task. A text statement that the agent has responded is not an actuation measurement.

Central claim to test. Managing action validity and commitment under changing observations can improve disturbed-task success and response-tail latency at matched compute, beyond merely increasing observation frequency or reducing model latency. Event-triggered execution, asynchronous chunking, and predictive control provide three technical routes to this claim. Each has an appropriate matched baseline; they need not be combined to demonstrate an effect.

Prior work and hypotheses. Real-Time Chunking (RTC) already studies asynchronous action-chunk execution. The proposed increment concerns invalidation by environmental change, measured physical response, and its relationship to final success. H1: genuine preemption improves recovery at matched sensing frequency. H2: delay- and state-conditioned action commitment improves over fixed chunking at matched inference budget. H3: calibrated short-horizon prediction improves delayed control, while prediction errors can cause regression. The paper must identify these boundaries instead of assuming that asynchronous or predictive systems are inherently more responsive.

2. Approach A: Event-Triggered, Preemptible Skills

Events and validity conditions. A skill declares observable preconditions: its target remains within a valid region, grasp state is unchanged, or the visible path remains compatible with the plan. A timestamped sensor stream feeds an event detector using change magnitude, confidence, and task phase. Small deviations may remain within the local controller’s responsibility. Larger changes trigger recovery or replanning, preventing every noisy image from interrupting stable motion.

Preemption protocol. Action packets carry a skill identifier, plan version, source-observation timestamp, expiry time, and cancellation marker. When an event invalidates a packet, the runtime revokes unexecuted commands and a single control owner stops or changes the motion and acknowledges the transition. A later model result based on an obsolete plan or observation must be revalidated before use. Cancelling a software wait does not stop the robot; logs must distinguish cancellation requested, accepted, and physically observed.

Recovery. Prefer local, verifiable responses such as slowing, holding, retracting briefly, or observing again. Invoke a slower Planner when the goal or program assumptions have changed. Debouncing, hysteresis, and cooldown reduce repeated triggers, but cooldown can also conceal important events and must be evaluated. Compare fixed thresholds with an event score learned on development data and frozen at test time. Ground-truth event times may support offline analyses but never drive the online detector.

Core comparison. Compare end-of-skill inspection, periodic inspection, and event-triggered execution with identical sensing rates. Measure false triggers, missed events, revoked commands, and stale actions that still execute. This separates better detection from actual preemptibility. Implement priority cancellation through the control owner or its queue, rather than allowing multiple threads to switch robot modes concurrently.

3. Approach B: Delay-Aware Asynchronous Action Chunking

The tradeoff. A continuous-action model predicts H future actions at once. Longer chunks reduce inference stalls but extend the influence of stale observations. Short chunks update more often, yet inference can outlast their execution and create gaps or discontinuities. The proposed method overlaps next-chunk inference with current execution and adapts the committed prefix to measured delay and scene change.

Alignment. With control interval Δt and estimated inference-plus-transfer delay $\hat{L}$, an initial commitment estimate is $p \approx \lceil \hat{L} / \Delta t \rceil$, refined by actions already executed and the device’s irrevocable buffer. A new chunk must join the actual execution state, not the state when inference was requested. Recent delay distributions support the estimate, with explicit behavior under unexpected tail latency.

Conditioning on change. A fixed prefix may be unnecessarily conservative in static scenes and too slow under target motion. Use target velocity, contact changes, and uncertainty to adjust the horizon and overlap. If the prefix becomes invalid, replace it at the cancellation boundaries supported by the device. Velocity or acceleration continuity constraints reduce abrupt transitions. The method must report any movement restrictions introduced by this commitment strategy.

Research increment. RTC is the main baseline. Compare fixed asynchronous chunks, delay-only adaptation, state-change-only adaptation, and their combination using the same action model and weights. This keeps the research centered on execution. If state-aware commitment does not improve final success or response tails, the conclusion should be limited to the measured continuity or utilization benefit.

4. Approach C: Predictive State Updates and Control

Predicting the state at actuation. Instead of using the latest target position directly, estimate the state when an action will take effect. Inputs include observation history, proprioception, candidate actions, and measured delay. A short-horizon model predicts future state and uncertainty. Receding-horizon optimization selects an action sequence, executes a prefix, and corrects predictions using new observations. Constant-velocity extrapolation and simple dynamics are explicit baselines before introducing a learned action-conditioned model.

Objective. Balance expected task cost, uncertainty, and control changes:

$$\min_{a_{t:t+H}} \mathbb{E}[C_{\text{task}}] + \lambda_U C_{\text{uncertainty}} + \lambda_A C_{\text{action}}.$$

High uncertainty may shorten the horizon, request another observation, or favor a conservative local action. The predictor uses permitted observations only. Simulator future states can measure offline prediction errors but are never supplied to the deployable planner.

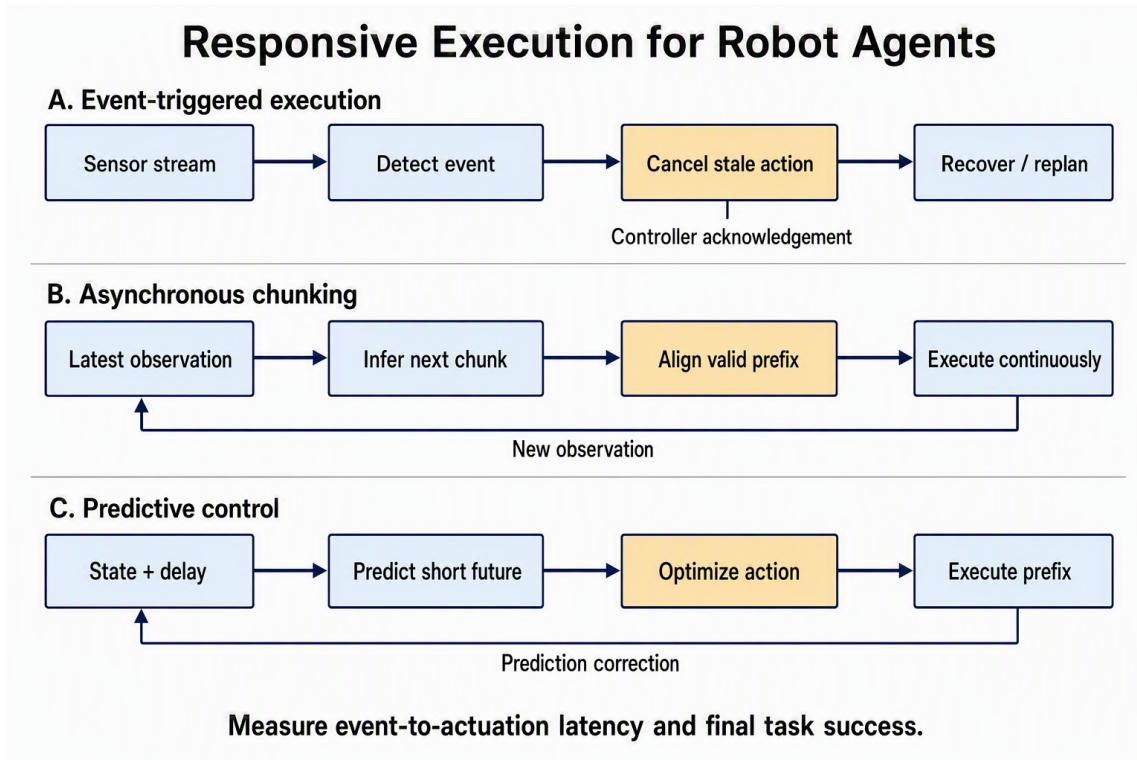


Figure 1: Independent pipelines for event-triggered execution, asynchronous chunking, and prediction. GPT-image-2 conceptual diagram. The evaluation target is physical correction and eventual task completion, not the time at which the model finishes generating a response.

Mechanism tests. Compare smooth target motion, sudden direction changes, brief occlusion, and contact-condition changes. Prediction may help under smooth motion and fail under abrupt changes; uncertainty calibration and observation correction should explain the boundary. Report both prediction accuracy and control performance. More accurate images or states do not automatically imply better actions, and excessive conservatism can reduce completion. Hardware timestamps must be aligned before making millisecond-scale claims.

5. Experimental Design: Dynamic Tasks and Delay Interventions

Perturbation panel. Extend LIBERO short tasks with four task-appropriate event classes: target movement, grasp/contact change, visible path-condition change, and instruction revision. Events occur before completion of the relevant action and leave a feasible task. For example, arbitrary teleportation of an already grasped target is not used as a natural-motion perturbation. An independent evaluator injects events, and policies discover them through declared sensing or instruction channels.

Proposed scale. Select 24 task–event pairs, covering each event class across multiple manipulation tasks. Three disturbance strengths, four delay conditions, and five seeds give 1,440 episodes per method. Pairs sharing an underlying task are not statistically independent. Separate development and validation panels determine solvability, magnitude, and timing before the test list is frozen. Adjust sample requirements using development variance rather than observed test effects.

Delay interventions. An initial design adds 0, 100, 300, or 600 ms of inference delay while recording actual total latency. Separate experiments add delay to sensing transfer or action submission; these are distinct interventions. A tail-latency panel compares distributions with similar means and different p95 values. If baseline inference already greatly exceeds the proposed bins, revise them against measured control and task timescales on validation, then freeze them.

Method	Update and execution	Mechanism isolated
Synchronous skill baseline	Observe after skill completion	Existing task-level feedback
Periodic inspection	Matched sensing, fixed update period	Value of observation frequency
Event preemption	Validity checking and acknowledged cancellation	Interruption and recovery
Short synchronous chunks	Frequent inference before execution	Resource cost of updates
RTC baseline	Asynchronous fixed-model chunks	Continuity under inference delay
State-aware chunking	Adaptive commitment and invalidation	Joint response to delay and change
Predictive control	Delay compensation and correction	Value of anticipating future state

Controlling confounds. Event methods share perception models and rates; chunking methods share action-model weights; predictive methods are compared under matched compute and movement limits. Record sampling rates, queue length, inference calls, and cumulative compute. Giving one method more GPUs or denser sensing does not establish a scheduling contribution. Report matched-budget comparisons and success–latency–cost tradeoffs where exact matching is impossible.

Benchmark boundaries. Report dynamic extensions separately from the standard 120-task short panel. The latter uses task-level pass@10 with spatial/object/goal and task/object/swap/lan breakdowns. Dynamic main tables report success on fixed perturbation instances and latency distributions. Exact manifests bind tasks, seeds, event trajectories, trigger rules, latency samples, and method versions. Preserve evidence and successful-seed resume protection. Cross-method trials are independent comparisons, while an unchanged method does not rerun an already successful task/seed configuration.

6. Planned Experiments: From Events to Recovery

Timing decomposition. Record physical event time t_e , the first sample containing the event t_s , detection t_d , corrective decision readiness t_p , controller acceptance t_c , and physical action change t_a . Report $t_s - t_e$, $t_d - t_s$, $t_p - t_d$, $t_c - t_p$, and $t_a - t_c$ where defined. A continuously updating method may have no discrete detection event; compare common measurable stages without inventing timestamps.

Defining effective response. Prespecify criteria by event: motion toward the changed target, grasp maintenance or recovery after slip, slowing/stopping/rerouting after obstruction, or ceasing actions that contradict a revised instruction. Validation freezes these rules, and the evaluator applies them offline. Observable physical change establishes response; a Planner declaration does not. Final task success remains an independent harness verdict.

Experiment A: event-triggered mechanisms. At matched sensing rates, ablate preemption, expiry checks, and learned versus fixed event thresholds. Measure false triggers, missed events, failed acknowledgements, and stale commands. Introduce noise and occlusion to test whether hysteresis generalizes. Accurate detection without physical correction points to the execution path; fast stopping without successful recovery supports only a narrower claim.

Experiment B: asynchronous execution and tails. Vary the mean, variance, and spikes of inference latency, then examine chunk length, committed prefix, state-change threshold, and transition smoothing. Primary measures are disturbed-task success, response p95, and continuity; report p50, p99, and control gaps as secondary measures. Strong p99 claims require enough events. Measure velocity and acceleration changes so that aggressive motions cannot masquerade as an unqualified responsiveness gain.

Experiment C: whether prediction improves control. Remove delay compensation, uncertainty penalties, and observation correction separately. Compare constant-velocity and learned prediction for smooth motion and sudden turns. Analyze relationships between prediction error and task failure. Any future-state upper-bound analysis remains offline, in a separate table, with no future ground truth exposed to the evaluated agent.

Experiment D: responsiveness versus cost. At fixed call or compute budgets, compare success, response p95, tokens, and wall time. Event triggering may save unnecessary model calls in static scenes. Asynchrony may improve utilization while increasing cumulative compute. Prediction may be more accurate but too slow to help. Evaluate end-to-end cost rather than only reduced time waiting for a language model.

Possible observation	Appropriate conclusion
Faster response and higher disturbed-task success	Supports effective responsiveness after matching sensing and compute.
Faster stopping but slower recovery or more failures	Improves the stop path, not necessarily dynamic manipulation.
Smooth chunks still execute stale actions after abrupt changes	Continuity improves while validity management remains a bottleneck.
Prediction helps smooth motion but hurts sudden turns	State the motion boundary and analyze correction and uncertainty.
Frequent periodic checks perform equally well	The event method's distinct benefit may be computational savings.

7. Statistical Protocol, Risks, and Paper Contribution

Statistics. Use paired bootstrap intervals clustered by underlying task, sharing event trajectories and latency samples across methods. Preregister final disturbed-task success and response p95; detection, recovery, and compute are explanatory metrics. Episodes that never respond cannot silently disappear from latency statistics. Report nonresponse rate, timeout threshold, and conditional latency quantiles; use censoring-aware analysis where appropriate. Otherwise, a method responding only on easy cases could appear artificially fast.

Physical execution evidence. A submitted command may wait while the robot executes an older buffer. Validate response through proprioception, controller acknowledgements, and synchronized timestamps. Quantify inter-device clock error. Keep simulator physical time separate from wall time, especially in accelerated simulation. Record frame production, arrival, and model-consumption times separately to expose observation staleness.

Principal risks. Event detectors can overfit task-specific signals, requiring task-family holdouts. Repeated interruptions can destabilize otherwise effective local control, requiring explicit cancellation boundaries and smooth transitions. Predictive models can fail abruptly, requiring calibration and regression analysis. The three approaches require different model or execution interfaces, so use matched baselines for each rather than forcing an unfair single ranking.

Outcome and efficiency reporting. Only simulator or independent-harness verdicts count as task success. Provider, transport, image, resource, and interrupted records are excluded from valid-verdict task denominators and reported separately. Also report implementation failures, nonresponses, protection events, and incomplete coverage. Log prompt/completion/total tokens, VLM and unmetered calls, wall time, and perception/action/recovery time. Preserve videos, trajectories, failures, and diagnostic logs. Learned models must be evaluated on matched and held-out tasks; lower training loss is not proof of improved response.

Expected contribution. This paper should establish an execution-time account of responsiveness: when event-triggered validity checks, adaptive action commitment, or short-horizon prediction can improve behavior under environmental change. Its methodological contribution is a comparison with matched sensing and compute; its evaluation contribution is the separation of event time, physical correction, recovery, and final completion. The strongest supported claim must depend on actual closed-loop behavior.

Independence. The study can use fixed skills or a frozen VLA and does not depend on learning a new force controller or training with GPT-guided DAgger. Candidate methods receive independent ablations under the common question. Task counts, perturbations, and delay bins are proposed experimental settings to freeze after validation, not completed measurements or guaranteed gains.

References

- [1] Real-Time Execution of Action Chunking Flow Policies. 2025. <https://arxiv.org/abs/2506.07339>
- [2] Diffusion Policy: Visuomotor Policy Learning via Action Diffusion. 2023. <https://arxiv.org/abs/2303.04137>
- [3] π_0 : A Vision-Language-Action Flow Model for General Robot Control. 2024. <https://arxiv.org/abs/2410.24164>
- [4] Code as Policies: Language Model Programs for Embodied Control. 2022. <https://arxiv.org/abs/2209.07753>
- [5] Noematrix Team. RoboRSI research preview. 2026. <https://lab.noematrix.ai/blog/2-roborsi-research-preview/>

GPT-Guided DAgger for VLA

Learning from Executable Code-as-Policy Corrections

Research Proposal III · Zimo Wen

September 2026

Abstract

Vision–language–action policies can execute efficiently but accumulate errors when their own actions lead to states absent from the demonstration distribution. A GPT-based robot agent can reason about such failures and synthesize a recovery program, yet using that agent on every deployment episode retains substantial inference cost. This proposal studies GPT as an approximate corrective expert within a DAgger-style learning loop. The VLA visits states, GPT inspects permitted observations, Code as Policy turns corrections into executable behavior, and aligned corrective actions are aggregated to train the VLA. The research compares dense correction, bounded expert intervention, and budgeted selective queries. Its central criterion is improved performance of the frozen VLA after GPT has been removed. The intended paper investigates the quality, distribution, and cost of executable supervision, rather than a new low-level controller or a permanently assisted agent.

1. Motivation and Standalone Research Hypothesis

Why the VLA needs corrections on its own trajectories. Behavioral cloning learns from demonstrations, but a deployed policy can create small grasp, placement, or sequencing errors that compound into unfamiliar states. Additional successful demonstrations from clean initial states may not cover those states. DAgger addresses this distribution mismatch by querying an expert on states induced by the learner and aggregating the resulting labels. Here, GPT is the proposed expert; DAgger is the data-collection and learning procedure, not the name of a GPT control mode.

What Code as Policy contributes. A natural-language criticism such as “the grasp is unstable; approach from the side” is not an action target for a VLA. Robot-RSI can convert grounded observations and proposed recoveries into programs over executable skills. This makes Code as Policy a candidate bridge from GPT’s correction to time-aligned action supervision. Existing action and controller interfaces are retained. The research contribution concerns how corrections become training data and improve the student, rather than adding force or torque outputs.

A motivating example. A VLA closes its gripper beside a cup and starts lifting empty fingers. GPT sees the current images, robot state, and visible execution history, identifies the failed grasp, and produces a short re-observe–reapproach–grasp program. Execution records the observations and actual commands during recovery. Those data train the VLA to handle similar states later. Whether the corrected episode succeeds and whether the trained VLA improves are different questions and require separate measurements.

Central hypothesis. Verified, executable GPT corrections collected on successive student-induced distributions improve a standalone VLA more than equal-budget offline demonstrations, text-only critique, or a single round of correction. The proposed paper must establish both that the teacher provides useful supervision and that the student acquires persistent capability. A hybrid system that succeeds only while GPT is available does not establish the latter.

Prior work and boundaries. DAgger, HG-DAgger, and SafeDAgger already establish iterative aggregation, expert intervention, and selective querying. Code as Policies already turns language reasoning into programs. The proposed increment is an auditable conversion of GPT-generated recovery programs into VLA-compatible supervision at learner-visited states, with teacher error and cost explicitly measured. GPT is an approximate expert; classical guarantees under stronger expert assumptions cannot simply be transferred to this setting.

2. Method: Executable Supervision on the Student Distribution

Actors and information. Let π_{θ_k} be the VLA at aggregation round k , E the GPT expert, and C the existing Code-as-Policy runtime. The expert receives the instruction, current permitted images, proprioception, and a bounded visible history h_t . It cannot read simulator target poses, hidden object state, success predicates, or future observations. Generated programs use a restricted set of declared sensing and skill calls; the same boundary applies to executed code, not just the prompt.

From correction to label. At a selected learner-visited state, $E(o_t, g, h_t)$ generates a correction program p_t . The runtime executes p_t from that actual state, recording the resulting sequence of observable histories and commands in the student’s action representation. A textual plan, intended waypoint, or VLA action merely judged “bad” is not an expert action label. For an action-chunk student, labels are contiguous executed expert segments with masks at intervention and handoff boundaries. Teacher future actions are never attached to observations from an incompatible student continuation.

Aggregation objective. Start with a declared demonstration dataset D_0 . Let D_k^E contain labeled corrective histories collected in round k and m_i mark executable, aligned label components. Optimize an appropriate supervised action loss, such as cloning or the student’s existing flow/diffusion objective:

$$\theta_{k+1} = \arg \min_{\theta} \left[\lambda_0 \mathbb{E}_{D_0} \ell_{\theta} + \sum_{j=1}^k \lambda_j \mathbb{E}_{D_j^E} [w_i m_i \ell_{\theta}(a_i^E \mid o_i, g, h_i)] \right].$$

The weights balance original skill retention, recent student states, and older corrections. Selection rules and weights are tuned only on development data. Unqueried student actions are not silently relabeled as expert demonstrations. GPT confidence alone is not a trustworthy label-quality score.

3. Three Correction Protocols within the Same Learning Loop

Approach A: dense state-conditioned correction. Query at predetermined action boundaries during learner rollouts and execute a short expert correction segment. This supplies a reference for the value of broad student-state coverage. Random query times with the same number of calls provide a budget-matched control. If the collector pauses simulation during GPT inference, it pauses at the queried state, records the pause, and charges wall-time cost; the result is a training protocol, not a real-time capability claim.

Approach B: bounded corrective takeover. A detector using visible progress, grasp evidence, or policy uncertainty requests GPT when the student appears to deviate. GPT temporarily takes over through a recovery program, then returns execution to the VLA after a declared observable condition or time limit. This is an intervention-based DAgger variant. Log the student-to-expert switch, expert segment, and handoff; longer teacher segments shift the collection distribution toward the expert and must be measured rather than described as purely student-generated experience.

Approach C: budgeted selective correction. A query rule estimates whether another expert label is useful from student uncertainty, novelty relative to the aggregated dataset, and expected correction benefit. Compare it with fixed-rate and randomly selected queries under the same call and execution budgets. A small randomized exploration quota samples apparently easy states and exposes blind spots. The teacher, label representation, and student-training procedure remain the same across protocols, isolating which states are queried.

4. Pipeline and the Correction-Data Contract

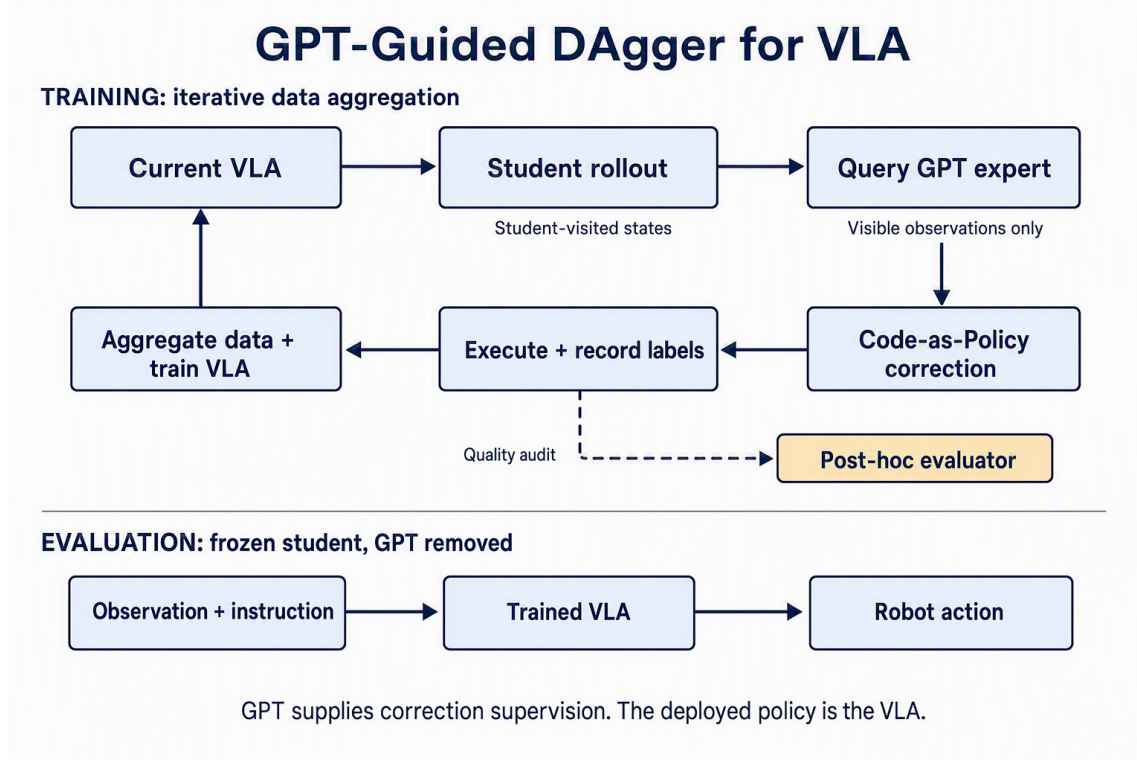


Figure 1: GPT supplies executable correction supervision during training. Student rollouts, corrective execution, aggregation, and retraining repeat across rounds. The lower band evaluates the frozen VLA with GPT removed. GPT-image-2 conceptual diagram. The post-hoc evaluator audits outcomes without exposing hidden state to the teacher.

Record structure. Each record binds task and round identifiers, student checkpoint, queried observation/history, expert program version, actual action segment, timestamps, control frame, action normalization, intervention boundaries, validity mask, and the later harness verdict. The evaluator’s hidden state stays outside teacher, student, Planner, Engineer, and Reviewer context. Post-hoc outcomes may support development-data audits and declared selection rules, never online access to hidden predicates.

What is validated. Check syntax and allowed calls, timestamp alignment, frame conversion, execution acknowledgements, and observable local effects. An eventual task success does not prove that every action was optimal. Conversely, an episode that later fails can contain a valid local correction. Retain failed and rejected traces, with reasons; compare quality filtering with all technically valid corrections to measure selection bias. Do not discard difficult states simply because the teacher failed to recover them.

Implementation boundary. The preferred collector executes corrections in the same environment state that triggered the query. Optional simulator branching uses evaluator-owned opaque checkpoints, exposes only permitted observations, and is reported separately. A simulated recovery from a modified or reset state cannot label the original state without an explicit alignment justification.

5. Experimental Design: Collection, Training, and Evaluation

Initial student and task separation. Start from one fixed VLA checkpoint and a declared D_0 , using existing Robot-RSI skill execution and trace collection. Select task families before collection, provisionally grouping 24 training instances, eight validation instances, and 16 held-out instances. Near-duplicate language or layout variants remain within one split; adjust counts rather than break family separation. Hold out object arrangements, physical perturbations, and designated task compositions, distinguishing unseen-task generalization from recovery on familiar tasks.

Aggregation schedule. An initial design uses three rounds, each with at most 200 collection episodes, 400 GPT queries, and a fixed expert-action duration cap. These are simultaneous ceilings, not guaranteed yields; report queries, accepted labels, expert control time, unique queried states, and completed verdicts. Freeze the GPT snapshot, prompt, runtime, and allowed skills across the main comparison. Each round deploys the newly trained student, while a fixed-student collection baseline isolates iterative distribution adaptation.

Policy mixing. The dense reference may use a declared expert/student mixing schedule, for example expert segment-selection probabilities of 0.5, 0.25, and 0 across rounds, while still querying student states when selected by the labeling protocol. The last round is student-driven except for explicitly logged corrective segments. Intervention-based methods report the realized expert-controlled fraction rather than claiming equivalence to canonical dense DAgger. All approaches must share an explicit collector protocol and budget.

Comparison	Additional data or assistance	Question isolated
Frozen initial VLA	None	Starting capability
More offline cloning	New expert rollouts from initial states	Data quantity versus visited-state relevance
One-round correction	One collected correction set	Iteration versus single fine-tuning
Fixed-student aggregation	Multiple rounds from the initial student	Value of updating the collection distribution
Text critique only	Critique, without executed expert targets	Whether language feedback supplies useful supervision
GPT-assisted hybrid	Expert remains available at evaluation	Online rescue versus learned student improvement
GPT-DAgger protocols	Dense, takeover, or selective correction	Query allocation and teacher supervision

Fair budgets. Match additional teacher execution time, accepted action-label count, and student gradient updates in separate controlled panels; one scalar rarely matches all three. Include a fixed-skill or human-expert correction baseline on a smaller matched subset to measure teacher quality. Text-only critique is a deliberately different feedback channel, not an equal-action-supervision expert. Hyperparameters, replay ratios, and checkpoint selection use validation only.

Frozen student tests. After every round, evaluate a frozen student without GPT on matched tasks and held-out tasks. A proposed held-out panel of 16 instances, four conditions, and five seeds yields 320 episodes per checkpoint. Use both natural initial-state rollouts and independently collected recovery-state probes; teacher-corrected training states are not held-out recovery tests. The standard 120-task LIBERO short panel separately reports task-level pass@10 with subset and suite breakdowns.

6. Planned Experiments and Evidence Required for the Claim

Experiment A: does correction become student capability? Compare the initial and round-1/2/3 students with GPT disabled, using matched seeds and held-out tasks. Plot simulator-confirmed success against cumulative labels, expert control time, GPT calls, and training compute. Report the GPT-assisted hybrid in a separate panel. Improvement confined to assisted execution supports an online rescue system, not successful DAgger-based transfer into the VLA. Training loss alone is insufficient.

Experiment B: why aggregate iteratively? Compare fresh student-induced collection with equal-size offline demonstrations, one-round correction, and repeated collection from the initial student. Keep student update counts and replay budgets controlled. Measure whether successive rounds cover new error states and reduce recurrence of previously corrected failure modes. If fixed-student or initial-state data achieve the same result, the on-distribution aggregation claim must be narrowed.

Experiment C: whether Code as Policy is the useful teacher interface. Hold queried states fixed and compare GPT text critique, direct numerical action proposals, fixed recovery skills, and executed GPT-generated programs. For same-state comparisons, use isolated evaluator-owned simulator branches where supported, with identical permitted observations and no state inspection by the agent. Report program validity, command execution, local correction success, label alignment, teacher wall time, and downstream student improvement. Extra runtime capability must not be confused with better reasoning.

Experiment D: teacher error and filtering. Compare all technically valid labels, declared observable-quality filtering, and post-hoc outcome-based filtering as distinct development-data variants. Include known alignment corruptions only as labeled diagnostic ablations. Measure teacher failure, label rejection by state difficulty, and the effect of retaining original demonstrations. A small human audit of permitted traces can estimate correction quality, but cannot silently repair one method’s training data. Successful-task filtering alone risks removing precisely the hard states DAgger is meant to cover.

Experiment E: query and intervention budgets. Compare random queries, fixed-rate queries, visible-failure triggers, and uncertainty/novelty selection. Match query counts and expert execution budgets. Vary takeover duration and replay weights to expose the tradeoff between obtaining a complete recovery and keeping data close to the student distribution. A selective method should improve learning per expert call without ignoring rare catastrophic or persistent failure modes.

Possible result	Supported conclusion
Standalone VLA improves on matched and held-out tasks	Supports persistent learning if teacher/data/compute controls also hold.
Hybrid improves but GPT-free student does not	Teacher rescue works; distillation or data alignment remains inadequate.
Equal-budget offline data match iterative correction	Student-distribution querying has not shown an independent advantage.
Selective queries match dense learning at lower cost	Supports efficient teacher allocation within the measured task distribution.
Aggressive filtering improves training metrics but harms hard tasks	Selection bias may remove difficult states; examine coverage and retention.

Primary endpoint and statistics. Preregister GPT-free held-out success improvement at a fixed expert budget. Use multiple student training seeds and paired evaluation seeds, with intervals clustered by task and variation across training runs reported separately. Validate sample size using development variance. Report forgetting on original tasks and inference efficiency alongside new recovery success.

7. Feasibility, Failure Modes, and Expected Paper Contribution

Robot-RSI integration. The VLA supplies student rollouts; existing Planner/Engineer capabilities generate and execute bounded correction programs; the trajectory store aggregates aligned action labels. Reviewer-visible evidence is restricted to allowed observations and receipts, while the independent harness supplies post-hoc task verdicts. Dataset construction records control ownership, teacher versions, and state provenance. Formal student evaluation disables teacher intervention and persistent write-back. No new force controller is required for this paper.

Partial observability and noisy expertise. GPT may misidentify objects, diagnose the wrong failure, or generate a recovery that is feasible but not useful. The student must receive the observable history needed to explain its labels. A teacher using privileged information unavailable to the student would create an unlearnable or unfair target; such access is excluded. Action ambiguity also means one valid correction need not be the unique optimal action. Model-appropriate sequence losses and multiple feasible trajectories may be preferable to treating every alternative as an error.

Latency and state drift. A delayed teacher response can refer to an obsolete state. A simulation collector may pause with cost accounting, while a physical collector must hold or resample safely and relabel from the actual observation at takeover. Never pair an old image with a correction executed after an unrecorded state change. This training-time issue is distinct from the real-time-control objective of the second paper.

Coverage, forgetting, and evidence. Corrections can overrepresent hard states or long teacher takeovers. Balance replay against D_0 , report the realized collection distribution, and evaluate retention. Preserve all rejected, failed, diagnostic, and interrupted records. Task success comes only from final simulator/harness verdicts; provider, transport, image, resource, and interrupted records are excluded from valid task denominators and reported separately, with implementation errors and missing coverage visible. Log teacher prompt/completion/total tokens, calls, unmetered calls, collection wall time, action/recovery time, and student training and inference cost.

Expected contribution. The paper should demonstrate a concrete learning interface between Code as Policy and VLA: GPT generates executable corrections, those corrections become aligned supervision on learner-visited states, and dataset aggregation improves a GPT-free student. Its evidence must isolate iteration, executable label quality, query allocation, and data scale. The novelty claim is this verified supervision mechanism and its measured learning value, not DAgger itself or continued reliance on GPT during deployment. A broader literature comparison is needed before any first-of-its-kind claim.

References

- [1] Ross, Gordon, and Bagnell. A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning. 2011. <https://arxiv.org/abs/1011.0686>
- [2] HG-DAgger: Interactive Imitation Learning with Human Experts. 2018. <https://arxiv.org/abs/1810.02890>
- [3] Query-Efficient Imitation Learning for End-to-End Autonomous Driving. 2016. <https://arxiv.org/abs/1605.06450>
- [4] Code as Policies: Language Model Programs for Embodied Control. 2022. <https://arxiv.org/abs/2209.07753>
- [5] π_0 : A Vision-Language-Action Flow Model for General Robot Control. 2024. <https://arxiv.org/abs/2410.24164>
- [6] Noematrix Team. RoboRSI research preview. 2026. <https://lab.noematrix.ai/blog/2-roborsi-research-preview/>